

Project: The Grid-Eye

Infrastructure Sentinel v1.0

Lucas De-Zarraga

July 15, 2026

1 Introduction

The Infrastructure Sentinel is a real-time monitoring dashboard designed for power grid management. It utilizes the PDCurses library for high-fidelity terminal rendering.

2 System Configuration

The project was configured using a TDM-GCC-64 environment. Key technical hurdles included bridging the Windows Entry Point (`WinMain`) and static linking of the PDCurses library.

3 Initial ASCII Interface

The current build features a multi-station status report:

```
=====
||      INFRASTRUCTURE SENTINEL      ||
=====
|Node|-----|Load|-----|Status|
-----
1. [MasTec HQ   ] [ 51%]      OK
2. [Google HQ   ] [ 53%]      OK
3. [Intel HQ    ] [ 77%]      OK
4. [Apple HQ    ] [ 48%]      OK
5. [HP HQ       ] [ 48%]      OK
-----
```

3.1 Technical Hurdles

Initial compilation failed due to a conflict between the standard console `main()` and the Windows GUI `WinMain()` entry points. This was resolved by:

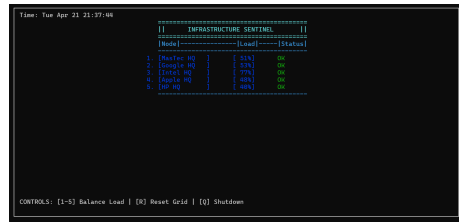


Figure 1: This is the final product.

- Utilizing the `-mconsole` linker flag.
- Implementing static linking with `-static` to avoid DLL dependencies.
- Ensuring the library path was explicitly defined for the TDM-GCC compiler.

4 Setting up the UI

Currently, the build features a prompt to let users deal with the nodes.

4.1 Technical Issues

When resizing the terminal, the UI would become distorted. To fix this, To fix this, I enabled the keypad and handled the resize signal within the main loop:

```

\\Outside main loop
keypad(stdscr, TRUE);
//Within main loop
if(c == KEY_RESIZE){ resize_term(0,0); }

```

4.2 Spatial Alignment

To ensure that the UI is properly aligned, I passed the `startX` variable into the `drawGrid()` function's integer `startX` argument.

5 Game play loop

The loop goes as follows: The load increases, and you, the user, must press either a number on your keyboard/keypad that corresponds to the node's number to reset the node or press "r" to reset the entire grid.

This is what the final product looks like:

5.1 Features

The following are features of the code:

- Inputs have cooldowns, so time your inputs correctly.
- The loads update every 0.5 seconds.
- There is a 1/100 chance for a power surge, so be on the lookout for sudden load spikes.
- If the load is 100%, it's game over!

5.2 Challenges

One significant challenge was ensuring terminal stability during process termination. Failure to reach the `endwin()` function can lead to "zombie" terminal states. This was resolved by implementing robust exit conditions and ensuring proper resource cleanup. Furthermore, the UI utilizes dynamic coordinate calculation (`getmaxyx`) to maintain visual symmetry across different screen resolutions.

6 Conclusion

This project demonstrates proficiency in low-level systems programming, real-time data visualization, and user experience design for industrial applications.